MACHINE LEARNING LAB

All 15 Experiments – Questions with Simple Python Code

Common imports (use at the beginning of each program):

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import *
```

Install required libraries: pip install pandas numpy matplotlib seaborn scikit-learn scipy mlxtend scikit-fuzzy minisom

Important: Check each CSV's actual column names before running. Replace example names such as Glucose, Gender, Price, Temperature, Weather and Category if your file uses different names.

EXPERIMENT 1 – Diabetes.csv, Rampur.csv

Part 1 – Data analysis

Read data and display null values

```
df = pd.read_csv("Diabetes.csv")
print(df.head())
print(df.isnull().sum())
```

Five-number summary and box plot

```
print(df["Glucose"].describe())
sns.boxplot(x=df["Glucose"])
plt.show()
```

Bivariate categorical plot

```
sns.catplot(x="Outcome", y="Glucose", data=df)
plt.show()
```

Remove rows with Z-score outliers greater than 3

```
from scipy.stats import zscore
z = np.abs(zscore(df.select_dtypes(include=np.number)))
df = df[(z < 3).all(axis=1)]
print(df.shape)
```

Part 2 – Simple linear regression: predict Discharge from Wind

Regression model, predictions, regression line and metrics

```
df = pd.read_csv("Rampur.csv")
X = df[["Wind"]]
y = df["Discharge"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
print("Intercept:", model.intercept_)
print("Coefficient:", model.coef_)
y_pred = model.predict(X_test)
print("Actual:", y_test.values)
print("Predicted:", y_pred)
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
plt.scatter(X_test, y_test)
plt.plot(X_test, y_pred, color="red")
plt.show()
```

EXPERIMENT 2 – toyato.csv, Kasol.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("toyato.csv")
print(df.head())
```

Minimum, maximum and mode for numerical columns

```
num = df.select_dtypes(include=np.number)
```

```
print(num.min())
print(num.max())
print(num.mode().iloc[0])
```

Count numerical and categorical columns

```
print("Numerical:", len(df.select_dtypes(include=np.number).columns))
print("Categorical:", len(df.select_dtypes(exclude=np.number).columns))
```

Swarm plot

```
sns.swarmplot(y=df["Price"])
plt.show()
```

Mode imputation for categorical columns with >10% missing

```
for col in df.select_dtypes(exclude=np.number).columns:
    if df[col].isnull().mean() > 0.10:
        df[col] = df[col].fillna(df[col].mode()[0])
```

Part 2 – Multiple linear regression: predict Discharge

Fit model, display coefficients, predict and evaluate

```
df = pd.read_csv("Kasol.csv")
X = df.drop("Discharge", axis=1)
y = df["Discharge"]
X = pd.get_dummies(X, drop_first=True).fillna(0)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
y_pred = model.predict(X_test)
print("Actual:", y_test.values)
print("Predicted:", y_pred)
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.show()
```

EXPERIMENT 3 – Customers.csv, Solar.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("Customers.csv")
print(df.head())
```

Positive, negative and near-zero correlation

```
corr = df.select_dtypes(include=np.number).corr()
print("Positive:", corr[corr > 0.7])
print("Negative:", corr[corr < -0.7])
print("Near zero:", corr[(corr > -0.1) & (corr < 0.1)])
```

Bar/count plot for categorical feature

```
sns.countplot(x="Gender", data=df)
plt.show()
```

Drop columns with >10% missing values

```
missing = df.isnull().mean() * 100
df = df.drop(columns=missing[missing > 10].index)
```

Mean imputation for numerical columns with <10% missing

```
for col in df.select_dtypes(include=np.number).columns:
    if 0 < df[col].isnull().mean() < 0.10:
        df[col] = df[col].fillna(df[col].mean())
```

Part 2 – Logistic regression: classify Solar Radiation into two classes

Create binary target, train and evaluate

```
df = pd.read_csv("Solar.csv")
df["Category"] = (df["Solar Radiation"] > df["Solar Radiation"].median()).astype(int)
X = df.drop(["Solar Radiation", "Category"], axis=1)
y = df["Category"]
X = pd.get_dummies(X, drop_first=True).fillna(0)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1:", f1_score(y_test, y_pred, zero_division=0))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True)
plt.show()
```

EXPERIMENT 4 – Stores.csv, Diabetes.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("Stores.csv")
print(df.head())
```

Numerical columns: min, max and mean

```
num = df.select_dtypes(include=np.number)
print(num.min())
print(num.max())
print(num.mean())
```

Skewness and distribution plots

```
print(num.skew())
for col in num.columns:
    sns.histplot(df[col], kde=True)
    plt.show()
    if df[col].skew() > 0: print(col, "Right Skew")
    elif df[col].skew() < 0: print(col, "Left Skew")
    else: print(col, "No Skew")
```

Mean imputation where missing >10%

```
for col in num.columns:
    if df[col].isnull().mean() > 0.10:
        df[col] = df[col].fillna(df[col].mean())
```

Standard scaling

```
scaler = StandardScaler()
df[num.columns] = scaler.fit_transform(df[num.columns])
print(df.head())
```

Part 2 – Single-layer perceptron: classify Outcome

Train, predict, confusion matrix and metrics

```
df = pd.read_csv("Diabetes.csv")
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42)
from sklearn.linear_model import Perceptron
model = Perceptron()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1:", f1_score(y_test, y_pred, zero_division=0))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True)
plt.show()
```

EXPERIMENT 5 – Solar-Regression.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("Solar-Regression.csv")
print(df.head())
```

High positive and near-zero correlation

```
corr = df.select_dtypes(include=np.number).corr()
print(corr[corr > 0.7])
print(corr[(corr > -0.1) & (corr < 0.1)])
```

Scatter plot for two continuous variables

```
sns.scatterplot(x="Temperature", y="Solar Radiation", data=df)
plt.show()
```

Median imputation where missing >10%

```
for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().mean() > 0.10:
        df[col] = df[col].fillna(df[col].median())
```

Remove columns containing missing values

```
df = df.dropna(axis=1)
print(df.isnull().sum())
```

Part 2 – MLP regression: predict Solar Radiation

Train, predict and evaluate

```
df = pd.read_csv("Solar-Regression.csv")
X = df.drop("Solar Radiation", axis=1)
y = df["Solar Radiation"]
X = pd.get_dummies(X, drop_first=True).fillna(0)
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
from sklearn.neural_network import MLPRegressor
model = MLPRegressor(hidden_layer_sizes=(100, 50), max_iter=1000, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2:", r2_score(y_test, y_pred))
plt.scatter(y_test, y_pred)
plt.show()
```

Randomized Search CV and Grid Search CV

```
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV
params = {
    "hidden_layer_sizes": [(50,), (100,), (100, 50)],
    "activation": ["relu", "tanh"],
    "learning_rate_init": [0.001, 0.01]
}
random = RandomizedSearchCV(
    MLPRegressor(max_iter=1000, random_state=42),
    params, n_iter=4, cv=3, random_state=42)
random.fit(X_train, y_train)
print("Random Best:", random.best_params_)
grid = GridSearchCV(
    MLPRegressor(max_iter=1000, random_state=42), params, cv=3)
grid.fit(X_train, y_train)
print("Grid Best:", grid.best_params_)
```

EXPERIMENT 6 – Precipitation.csv, Diabetes.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("Precipitation.csv")
print(df.head())
```

Display outliers using Z-score

```
from scipy.stats import zscore
num = df.select_dtypes(include=np.number)
z = np.abs(zscore(num, nan_policy="omit"))
print(df[(z > 3).any(axis=1)])
```

Count plot between two categorical variables

```
sns.countplot(x="Gender", hue="Outcome", data=df)
plt.show()
```

Drop columns with variance <10

```
num = df.select_dtypes(include=np.number)
print(num.var())
df = df.drop(columns=num.var()[num.var() < 10].index)
```

Label encoding

```
le = LabelEncoder()
df["Category"] = le.fit_transform(df["Category"])
print(df.head())
```

Part 2 – Random Forest classification: classify Outcome

Train and evaluate

```
df = pd.read_csv("Diabetes.csv")
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.25, random_state=42)
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Optimize and compare

```
model2 = RandomForestClassifier(n_estimators=200, max_depth=5, random_state=42)
model2.fit(X_train, y_train)
pred2 = model2.predict(X_test)
print("Old:", accuracy_score(y_test, y_pred))
print("New:", accuracy_score(y_test, pred2))
plt.bar(["Old", "New"], [accuracy_score(y_test, y_pred),
                         accuracy_score(y_test, pred2)])
plt.show()
```

EXPERIMENT 7 – IRIS.csv, Diabetes.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("IRIS.csv")
print(df.head())
```

Negative and near-zero correlation

```
corr = df.select_dtypes(include=np.number).corr()
print(corr[corr < -0.7])
print(corr[(corr > -0.1) & (corr < 0.1)])
```

Pair plot

```
sns.pairplot(df, hue="Species")
plt.show()
```

One-hot encoding

```
df = pd.get_dummies(df, columns=["Species"])
print(df.head())
```

Pearson and Spearman correlation with heatmap

```
print(df.corr(numeric_only=True, method="pearson"))
print(df.corr(numeric_only=True, method="spearman"))
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.show()
```

Part 2 – MLP classification: classify Outcome

Train and evaluate

```
df = pd.read_csv("Diabetes.csv")
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
from sklearn.neural_network import MLPClassifier
model = MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=1000, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Optimize and compare

```
model2 = MLPClassifier(hidden_layer_sizes=(50,), max_iter=1500,
                       alpha=0.01, random_state=42)
model2.fit(X_train, y_train)
pred2 = model2.predict(X_test)
print("Old:", accuracy_score(y_test, y_pred))
print("New:", accuracy_score(y_test, pred2))
plt.bar(["Old", "New"], [accuracy_score(y_test, y_pred),
                         accuracy_score(y_test, pred2)])
plt.show()
```

EXPERIMENT 8 – Diabetes.csv, Iris.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("Diabetes.csv")
print(df.head())
```

Outliers using Z-score

```
from scipy.stats import zscore
num = df.select_dtypes(include=np.number)
z = np.abs(zscore(num, nan_policy="omit"))
print(df[(z > 3).any(axis=1)])
```

Violin plot

```
sns.violinplot(y=df["Glucose"])
plt.show()
```

Nearest interpolation

```
num = df.select_dtypes(include=np.number)
df[num.columns] = num.interpolate(method="nearest")
print(df.isnull().sum())
```

PCA

```
from sklearn.decomposition import PCA
X = df.select_dtypes(include=np.number).dropna()
X = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
result = pca.fit_transform(X)
print(result)
print(pca.explained_variance_ratio_)
```

Part 2 – Agglomerative clustering

Compare Euclidean/Manhattan and single/complete linkage

```
df = pd.read_csv("Iris.csv")
X = df.select_dtypes(include=np.number)
X = StandardScaler().fit_transform(X)
from sklearn.cluster import AgglomerativeClustering
for metric in ["euclidean", "manhattan"]:
    for link in ["single", "complete"]:
        model = AgglomerativeClustering(n_clusters=3, metric=metric, linkage=link)
        labels = model.fit_predict(X)
        print(metric, link)
        print("Silhouette:", silhouette_score(X, labels))
        print("DB Index:", davies_bouldin_score(X, labels))
```

Dendrogram

```
from scipy.cluster.hierarchy import dendrogram, linkage
Z = linkage(X, method="single")
dendrogram(Z)
plt.show()
# Use method="complete" for complete linkage.
```

EXPERIMENT 9 – IRIS.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("IRIS.csv")
print(df.head())
```

Five-number summary and box plot

```
print(df["SepalLengthCm"].describe())
sns.boxplot(x=df["SepalLengthCm"])
plt.show()
```

Pair plot

```
sns.pairplot(df, hue="Species")
plt.show()
```

Min-Max normalization

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[["SepalLengthCm"]] = scaler.fit_transform(df[["SepalLengthCm"]])
print(df.head())
```

LDA feature extraction

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X = df.select_dtypes(include=np.number)
y = df["Species"]
lda = LinearDiscriminantAnalysis(n_components=2)
result = lda.fit_transform(X, y)
print(result)
```

Part 2 – Fuzzy C-Means clustering

Preprocess and find clusters, centroids, labels and metrics

```
import skfuzzy as fuzz
df = pd.read_csv("IRIS.csv")
X = df.select_dtypes(include=np.number).dropna()
X = StandardScaler().fit_transform(X)
data = X.T
for n in range(2, 6):
    centers, u, _, _, _, _, fpc = fuzz.cluster.cmeans(
        data, c=n, m=2, error=0.005, maxiter=1000, init=None)
    labels = np.argmax(u, axis=0)
    print("Clusters:", n)
    print("Centroids:", centers)
    print("Labels:", labels)
    print("Partition Coefficient:", fpc)
    entropy = -np.mean(np.sum(u * np.log(u + 1e-10), axis=0))
    print("Classification Entropy:", entropy)
    plt.scatter(X[:, 0], X[:, 1], c=labels)
    plt.scatter(centers[:, 0], centers[:, 1], marker="X")
    plt.show()
```

EXPERIMENT 10 – Solar.csv, Iris.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("Solar.csv")
print(df.head())
```

Five-number summary and box plot

```
print(df["Temperature"].describe())
sns.boxplot(x=df["Temperature"])
plt.show()
```

Standard scaling

```
num = df.select_dtypes(include=np.number)
scaler = StandardScaler()
df[num.columns] = scaler.fit_transform(num)
print(df.head())
```

Skewness and distribution plots

```
print(df.skew(numeric_only=True))
for col in df.select_dtypes(include=np.number).columns:
    sns.histplot(df[col], kde=True)
    plt.show()
    if df[col].skew() > 0: print(col, "Right Skew")
    elif df[col].skew() < 0: print(col, "Left Skew")
    else: print(col, "No Skew")
```

Z-score normalization

```
from scipy.stats import zscore
df["Temperature"] = zscore(df["Temperature"])
print(df["Temperature"])
```

Part 2 – SOM clustering

Preprocess Iris data

```
df = pd.read_csv("Iris.csv")
X = df.select_dtypes(include=np.number).dropna()
X = StandardScaler().fit_transform(X)
```

Train SOM and display winning nodes/weights

```
from minisom import MiniSom
som = MiniSom(5, 5, X.shape[1], sigma=1.0, learning_rate=0.5)
som.random_weights_init(X)
som.train_random(X, 100)
labels = [som.winner(row) for row in X]
print("Cluster Labels:", labels)
centers = som.get_weights().reshape(-1, X.shape[1])
print("Centroids:", centers)
plt.scatter(X[:, 0], X[:, 1])
plt.show()
```

Cluster validation example using K-Means labels

```
from sklearn.cluster import KMeans
model = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = model.fit_predict(X)
print("Silhouette:", silhouette_score(X, labels))
print("DB Index:", davies_bouldin_score(X, labels))
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.show()
```

EXPERIMENT 11 – Diabetes.csv, Supermarket.csv

Part 1 – Data analysis

Read data and show null values

```
df = pd.read_csv("Diabetes.csv")
print(df.head())
print(df.isnull().sum())
```

Five-number summary and box plot

```
print(df["Glucose"].describe())
sns.boxplot(x=df["Glucose"])
plt.show()
```

Count plot

```
sns.countplot(x="Outcome", data=df)
plt.show()
```

SelectKBest

```
from sklearn.feature_selection import SelectKBest, f_classif
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
selector = SelectKBest(score_func=f_classif, k=3)
X_new = selector.fit_transform(X, y)
print(X.columns[selector.get_support()])
```

SelectPercentile

```
from sklearn.feature_selection import SelectPercentile
selector = SelectPercentile(score_func=f_classif, percentile=50)
X_new = selector.fit_transform(X, y)
print(X.columns[selector.get_support()])
```

Part 2 – Apriori association rules

Generate frequent itemsets and rules (10% support, 70% confidence)

```
from mlxtend.frequent_patterns import apriori, association_rules
df = pd.read_csv("Supermarket.csv")
data = pd.get_dummies(df.astype(str))
frequent = apriori(data, min_support=0.10, use_colnames=True)
print("Frequency Set:")
print(frequent)
rules = association_rules(frequent, metric="confidence", min_threshold=0.70)
print("Association Rules:")
print(rules)
print("High Confidence Rules:")
print(rules[rules["confidence"] > 0.70])
```

EXPERIMENT 12 – Diabetes.csv, house_price.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("Diabetes.csv")
print(df.head())
```

Outliers using Z-score

```
from scipy.stats import zscore
num = df.select_dtypes(include=np.number)
z = np.abs(zscore(num, nan_policy="omit"))
print(df[(z > 3).any(axis=1)])
```

Categorical plot

```
sns.catplot(x="Outcome", y="Glucose", data=df)
plt.show()
```

Mode imputation for categorical columns with >10% missing

```
for col in df.select_dtypes(exclude=np.number).columns:
    if df[col].isnull().mean() > 0.10:
        df[col] = df[col].fillna(df[col].mode()[0])
```

Z-score normalization and scatter plot

```
df["Glucose_Z"] = zscore(df["Glucose"])
plt.scatter(df.index, df["Glucose_Z"])
plt.show()
```

Part 2 – Random Forest regression: four parameter sets

Train four models and compare MSE/RMSE/R2

```
df = pd.read_csv("house_price.csv")
X = df.drop("Price", axis=1)
y = df["Price"]
X = pd.get_dummies(X, drop_first=True).fillna(0)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
from sklearn.ensemble import RandomForestRegressor
m1 = RandomForestRegressor(n_estimators=50, random_state=42)
m2 = RandomForestRegressor(n_estimators=100, max_depth=5, random_state=42)
m3 = RandomForestRegressor(n_estimators=150, max_depth=10, random_state=42)
m4 = RandomForestRegressor(n_estimators=200, max_depth=15, random_state=42)
models = [m1, m2, m3, m4]
for i, model in enumerate(models):
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print("Model", i+1)
    print("MSE:", mean_squared_error(y_test, pred))
    print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
    print("R2:", r2_score(y_test, pred))
```

Select best model and predict

```
best_model = min(models, key=lambda m: mean_squared_error(
    y_test, m.predict(X_test)))
y_pred = best_model.predict(X_test)
print("Best Model:", best_model)
print("Predicted Values:", y_pred)
```

EXPERIMENT 13 – Diabetes.csv, Iris.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("Diabetes.csv")
print(df.head())
```

Missing-value percentage and drop columns >20%

```
missing = df.isnull().mean() * 100
print(missing)
df = df.drop(columns=missing[missing > 20].index)
```

Scatter plot

```
sns.scatterplot(x="Glucose", y="BMI", data=df)
plt.show()
```

Label encoding

```
le = LabelEncoder()
df["Category"] = le.fit_transform(df["Category"])
print(df.head())
```

LDA feature extraction

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X = df.select_dtypes(include=np.number).drop(columns=["Outcome"])
y = df["Outcome"]
lda = LinearDiscriminantAnalysis(n_components=1)
result = lda.fit_transform(X, y)
print(result)
```

Part 2 – AdaBoost classification: four parameter sets

Train, compare and display confusion matrix and metrics

```
df = pd.read_csv("Diabetes.csv")
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
X = X.fillna(X.mean())
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
from sklearn.ensemble import AdaBoostClassifier
m1 = AdaBoostClassifier(n_estimators=50, learning_rate=0.1, random_state=42)
m2 = AdaBoostClassifier(n_estimators=100, learning_rate=0.5, random_state=42)
m3 = AdaBoostClassifier(n_estimators=150, learning_rate=1.0, random_state=42)
m4 = AdaBoostClassifier(n_estimators=200, learning_rate=0.01, random_state=42)
models = [m1, m2, m3, m4]
for i, model in enumerate(models):
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print("Model", i+1)
    print(confusion_matrix(y_test, pred))
    print("Accuracy:", accuracy_score(y_test, pred))
    print("Precision:", precision_score(y_test, pred, zero_division=0))
    print("Recall:", recall_score(y_test, pred, zero_division=0))
    print("F1:", f1_score(y_test, pred, zero_division=0))
```

Find best model

```
best_model = max(models, key=lambda m: accuracy_score(
    y_test, m.predict(X_test)))
print("Best Model:", best_model)
print("Best Prediction:", best_model.predict(X_test))
```

EXPERIMENT 14 – Diabetes.csv, Iris.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("Diabetes.csv")
print(df.head())
```

Five-number summary and box plot

```
print(df["Glucose"].describe())
sns.boxplot(x=df["Glucose"])
plt.show()
```

Count plot between categorical variables

```
sns.countplot(x="Gender", hue="Outcome", data=df)
plt.show()
```

Binarize numerical feature

```
df["Glucose_Binary"] = (df["Glucose"] > 100).astype(int)
print(df.head())
```

Remove highly correlated features

```
corr = df.select_dtypes(include=np.number).corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
drop_cols = [col for col in upper.columns if any(upper[col] > 0.8)]
df = df.drop(columns=drop_cols)
print(df.head())
```

Part 2 – Perceptron vs Logistic Regression

Classify two Iris species and compare models

```
df = pd.read_csv("Iris.csv")
df = df[df["Species"].isin(df["Species"].unique()[:2])]
X = df.select_dtypes(include=np.number)
y = LabelEncoder().fit_transform(df["Species"])
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42)
from sklearn.linear_model import Perceptron, LogisticRegression
p = Perceptron()
p.fit(X_train, y_train)
pred1 = p.predict(X_test)
print("Perceptron Accuracy:", accuracy_score(y_test, pred1))
print(confusion_matrix(y_test, pred1))
lr = LogisticRegression()
lr.fit(X_train, y_train)
pred2 = lr.predict(X_test)
print("Logistic Accuracy:", accuracy_score(y_test, pred2))
print(confusion_matrix(y_test, pred2))
print(classification_report(y_test, pred1))
print(classification_report(y_test, pred2))
plt.bar(["Perceptron", "Logistic Regression"],
        [accuracy_score(y_test, pred1), accuracy_score(y_test, pred2)])
plt.show()
```

EXPERIMENT 15 – Solar.csv

Part 1 – Data analysis

Read data

```
df = pd.read_csv("Solar.csv")
print(df.head())
```

Numerical columns: min, max and mean

```
num = df.select_dtypes(include=np.number)
print(num.min())
print(num.max())
print(num.mean())
```

Min-Max normalization and scatter plot

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[["Temperature"]] = scaler.fit_transform(df[["Temperature"]])
plt.scatter(df.index, df["Temperature"])
plt.show()
```

Label encoding

```
le = LabelEncoder()
df["Weather"] = le.fit_transform(df["Weather"])
print(df.head())
```

Remove low-variance features

```
from sklearn.feature_selection import VarianceThreshold
X = df.select_dtypes(include=np.number)
selector = VarianceThreshold(threshold=0.01)
result = selector.fit_transform(X)
print(X.columns[selector.get_support()])
```

Part 2 – MLP regression: predict Solar Radiation

Train, predict and evaluate

```
df = pd.read_csv("Solar.csv")
X = df.drop("Solar Radiation", axis=1)
y = df["Solar Radiation"]
X = pd.get_dummies(X, drop_first=True).fillna(0)
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
from sklearn.neural_network import MLPRegressor
model = MLPRegressor(hidden_layer_sizes=(100, 50), max_iter=1000, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2:", r2_score(y_test, y_pred))
plt.plot(y_test.values, label="Actual")
plt.plot(y_pred, label="Predicted")
plt.legend()
plt.show()
```

Randomized Search CV and optimal parameters

```
from sklearn.model_selection import RandomizedSearchCV
params = {
    "hidden_layer_sizes": [(50,), (100,), (100, 50)],
    "activation": ["relu", "tanh"],
    "learning_rate_init": [0.001, 0.01]
}
search = RandomizedSearchCV(
    MLPRegressor(max_iter=1000, random_state=42),
    params, n_iter=4, cv=3, random_state=42)
search.fit(X_train, y_train)
print("Best Parameters:", search.best_params_)
best_model = search.best_estimator_
pred = best_model.predict(X_test)
print("Old RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("New RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
```

QUICK REVISION – IMPORTANT METRICS

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, y_pred, zero_division=0))
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2:", r2_score(y_test, y_pred))
print("Silhouette:", silhouette_score(X, labels))
print("DB Index:", davies_bouldin_score(X, labels))
```

Remember: Classifier models are used for categories; Regressor models are used for continuous numerical predictions. Verify dataset column names before execution. The SOM validation example uses K-Means labels as a separate clustering validation example, not SOM labels.